

ISTANBUL AYDIN UNIVERSITY



JANUARY, 2026

BATUHAN TOMBAŞ

FACULTY OF APPLIED SCIENCES

SOFTWARE DEVELOPMENT

I. INTRODUCTION

1.1 Problem Statement

Hearing impairment affects millions of people worldwide, creating significant challenges in situational awareness and personal safety. Essential environmental cues—such as fire alarms, doorbells, crying babies, or approaching sirens—are often missed, leading to potential safety hazards and a reduced sense of independence. While visual indicators exist (e.g., flashing doorbells), they are location-specific and not portable

1.2 Proposed Solution

SoundRadar is a mobile Android application designed to effectively function as a "digital ear." Utilizing on-device Machine Learning (TensorFlow Lite), it continuously monitors environmental audio in real-time, classifies distinct sounds, and converts them into visual and haptic feedback. This ensures that users are alerted to critical events regardless of their location, functioning completely offline to ensure privacy and reliability.

II. SYSTEM ARCHITECTURE

The application follows the MVVM (Model-View-ViewModel) architectural pattern to ensure separation of concerns, testability, and maintainability.

2.1 Component Interaction User Layer:

MainActivity (UI) observes MainViewModel for real-time changes.

Service Layer: SoundMonitoringService runs eternally in the foreground, orchestrating the ML inference and Alert logic. Logic Layer:

AudioClassificationHelper bridges the raw hardware Audio input and the TFLite Interpreter. Data Layer: AppDatabase (Room) provides persistent storage for the Event History.

III. DEEP TECHNICAL IMPLEMENTATION

3.1 Audio Classification Engine (AudioClassificationHelper.java)

The core of SoundRadar relies on the YAMNet (Yet Another Mobile Network) architecture, a deep neural network that predicts 521 audio event classes. Initialization: The AudioClassifier is built with a BaseOptions configuration using 2 CPU threads for optimal battery performance. Input

Tensor: A TensorAudio buffer manages the incoming audio stream. It converts raw PCM data from AudioRecord into the specific float array format required by the model. Inference Loop: A ScheduledThreadPoolExecutor triggers classification every 900ms (0.9 seconds). Thresholding: A custom logic filters results. The raw probability threshold is set to 0.1 (10%) initially to capture faint sounds, but is further filtered dynamically by user sensitivity settings (0-100% slider maps to 0.1-1.0 confidence).

3.2 Intelligent Alert System (AlertManager.java)

The app employs a haptic compositing engine to convey urgency without looking at the screen.

Emergency Pattern (Fire, Siren):

Vibration: Uses VibrationEffect.createWaveform with a "Heavy-Heavy" pattern: {0, 500, 200, 500, 200, 1000} (ms). The amplitudes alternate between 0 (off) and 255 (max intensity). Visual: A strobe light effect is achieved by toggling the Camera Flash (setTorchMode) every 100ms via a Handler loop.

Warning Pattern (Doorbell, Dog):

Vibration: A single "OneShot" vibration of 500ms using VibrationEffect.DEFAULT_AMPLITUDE. Visual: No strobe, to avoid annoyance.

3.3 Deep Sleep Mode Algorithm (SoundMonitoringService.java)

To solve the "Alert Fatigue" problem, a time-window algorithm filters unnecessary notifications.

Logic:

The `isInDeepSleepWindow()` method calculates the current time against user preferences.

Cross-Midnight Handling:

The algorithm correctly handles overnight schedules (e.g., Start 23:00, End 07:00). Formula: `if (start < end) { return now >= start && now < end; } else { return now >= start || now < end; }`

Critical Override:

Even inside the Deep Sleep window, the system checks the label. If `label.contains("fire")` or `label.contains("siren")`, the filter is bypassed.

3.4 Data Persistence Strategy (`AppDatabase.java`)

SoundRadar uses Room Persistence Library over raw SQLite for type safety.

Schema:

events table (id, label, confidence, timestamp, is_critical).

Concurrency:

All database writes (insert) occur on a dedicated background ExecutorService with a fixed thread pool of 4, ensuring the UI thread never freezes even during high-frequency writes.

IV. KEY FEATURES AND USER EXPERIENCE

4.1 Real-Time "Radar" Visualization

The main screen features a pulsing radar animation (MainActivity.java) synchronized with the service state. This gives users immediate visual confirmation that the app is "listening," reducing anxiety about whether the tool is working.

4.2 Customizable Sound Filtering

Users can fine-tune the detection list via SoundFilterActivity . This is powered by a SharedPreferences set (disabled_sounds). The inference loop checks this "blocklist" before triggering any action, ensuring that a user who hates "Keyboard Typing" notifications never receives them.

V. TECHNICAL SPECIFICATIONS

Platform: Android (Min SDK 24, Target SDK 34).

Language: Java.

ML Framework: TensorFlow Lite Support Library
(org.tensorflow.lite.support).

Audio Source: android.media.AudioRecord (16kHz Sample Rate).
Concurrency: java.util.concurrent (ScheduledThreadPoolExecutor).
Permissions: RECORD_AUDIO, CAMERA (Flashlight),
POST_NOTIFICATIONS, FOREGROUND_SERVICE_MICROPHONE.

VI. CONCLUSION

SoundRadar demonstrates how on-device AI can be leveraged to solve real-world accessibility challenges. By converting auditory information into visual and tactile signals, it provides a vital layer of safety and awareness for hearing-impaired users. The architecture prioritizes offline reliability, battery efficiency (via threaded inference), and user customization (Deep Sleep/Filters).

VII. SOURCES

"AudioRecord | Android Developers", Google Developers,
<https://developer.android.com/reference/android/media/AudioRecord>,

"Save data in a local database using Room", Google Developers,
<https://developer.android.com/training/data-storage/room>,

"Sound classification with YAMNet", TensorFlow Hub,
<https://www.tensorflow.org/hub/tutorials/yamnet>,

"Guide to app architecture", Google Developers,
<https://developer.android.com/topic/architecture>,

"TensorFlow Lite for Android", TensorFlow,
<https://www.tensorflow.org/lite/android>,

"VibrationEffect | Android Developers", Google Developers,
<https://developer.android.com/reference/android/os/VibrationEffect>,

